

Практическое занятие 24: CRUD-операции с базой данных в Django

Цель занятия:

- Научиться реализовывать основные операции с данными: создание, чтение, обновление и удаление (CRUD).
 - Познакомиться с механизмами обработки запросов в Django и работой с формами.
-

Задачи занятия

1. Введение в CRUD-операции
 2. Реализация операции создания (Create)
 3. Реализация операции чтения (Read)
 4. Реализация операции обновления (Update)
 5. Реализация операции удаления (Delete)
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Создание своего мероприятия
 8. Выполнение дополнительных заданий. Массовое удаление, подтверждение, импорт/экспорт данных
-

Задача 1: Введение в CRUD-операции

1.1. Что такое CRUD-операции?

- **CRUD** — аббревиатура от Create (Создать), Read (Прочитать), Update (Обновить), Delete (Удалить).
- Это базовые операции, которые используются для работы с данными в большинстве веб-приложений.
- Позволяют пользователям взаимодействовать с базой данных через интерфейс приложения.

1.2. Почему CRUD-операции важны?

- **Управление данными:** Позволяют создавать новые записи, просматривать существующие, вносить изменения и удалять ненужные данные.

- **Пользовательский опыт:** Предоставляют интерфейс для взаимодействия с приложением, делая его полезным и функциональным.
- **Архитектура приложения:** CRUD-операции являются фундаментальными для построения RESTful API и MVC-паттернов.

Задача 2: Реализация операции создания (Create)

2.1. Создание формы `EventForm`

Шаги:

1. Создайте файл `forms.py` в приложении `events`, если его еще нет.
2. Добавьте код для формы:

```
# events/forms.py

from django import forms
from .models import Event

class EventForm(forms.ModelForm):
    class Meta:
        model = Event
        fields = ['title', 'description', 'date', 'location', 'category']
        widgets = {
            'date': forms.DateTimeInput(attrs={'type': 'datetime-local'}),
        }
```

Пояснение:

- **Импортируем `forms` и `Event`:** Используем встроенные формы Django и модель `Event`, которую мы создали ранее.
- **`EventForm`:** Наследуется от `forms.ModelForm`, что упрощает создание формы на основе модели.
- **`Meta` класс:**
 - `model`: Указывает модель, с которой связана форма.
 - `fields`: Определяет, какие поля будут включены в форму.
 - `widgets`: Позволяет настроить виджет для поля `date`, чтобы использовать HTML5-элемент `datetime-local`.

2.2. Создание представления `add_event`

Шаги:

1. Откройте файл `views.py` в приложении `events`.
2. Импортируйте необходимые модули:

```
# events/views.py

from django.shortcuts import render, redirect
from .forms import EventForm
from django.contrib.auth.decorators import login_required
from django.contrib import messages
```

3. Добавьте функцию `add_event`:

```
# events/views.py

@login_required
def add_event(request):
    if request.method == 'POST':
        form = EventForm(request.POST)
        if form.is_valid():
            event = form.save()
            messages.success(request, 'Мероприятие успешно добавлено.')
            return redirect('events:event_detail', event_id=event.id)
    else:
        form = EventForm()
    return render(request, 'events/add_event.html', {'form': form})
```

Пояснение:

- `@login_required`: Декоратор, который требует авторизации пользователя для доступа к этому представлению.
- **Обработка метода POST**:
 - Если форма отправлена, проверяем ее на валидность.
 - Если форма валидна, сохраняем новое мероприятие и перенаправляем пользователя на страницу деталей мероприятия.
- **Обработка метода GET**:
 - Если страница запрошена впервые, отображаем пустую форму для заполнения.
- Используем `messages` для отображения уведомлений пользователю.

2.3. Создание шаблона `add_event.html`

Шаги:

1. Создайте файл `add_event.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/add_event.html -->

{% extends 'events/base.html' %}

{% block title %}Добавить мероприятие{% endblock %}

{% block content %}
    <h1>Добавить мероприятие</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Сохранить</button>
    </form>
    <a href="{% url 'events:event_list' %}">Назад к списку мероприятий</a>
{% endblock %}
```

Пояснение:

- `{% extends 'events/base.html' %}`: Шаблон наследуется от базового.
- `{% csrf_token %}`: Токен защиты от CSRF-атак, обязателен для форм с методом POST.
- `{{ form.as_p }}`: Отображает форму с использованием тегов `<p>`.
- **Ссылка на список мероприятий**: Позволяет пользователю вернуться к списку.

2.4. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `add_event`:

```
# events/urls.py

from django.urls import path
from . import views

urlpatterns = [
    # ... другие маршруты ...
    path('events/add/', views.add_event, name='add_event'),
]
```

Задача 3: Реализация операции чтения (Read)

3.1. Создание представления `event_detail`

Шаги:

1. Откройте файл `views.py`.
2. Импортируйте необходимые модули:

```
# events/views.py

from django.shortcuts import render, get_object_or_404
from .models import Event
```

3. Отредактируйте функцию `event_detail`:

```
# events/views.py

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    comments = event.comments.all().order_by('-created_at')
    return render(request, 'events/event_detail.html', {
        'event': event,
        'comments': comments
    })
```

Пояснение:

- `get_object_or_404`: Получает объект по ID или возвращает 404 ошибку, если объект не найден.
- **Получаем комментарии к мероприятию:** Используем связь `event.comments.all()`.

3.2. Редактирование шаблона `event_detail.html`

Шаги:

1. Отредактируйте файл `event_detail.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'events/base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
```

```

<h1>{{ event.title }}</h1>
<p><strong>Дата и время:</strong> {{ event.date|date:"d.m.Y H:i" }}</p>
<p><strong>Место:</strong> {{ event.location }}</p>
<p><strong>Категория:</strong> {{ event.category.name }}</p>
<p>{{ event.description }}</p>

<h2>Отзывы</h2>
{% if event.reviews.exists %}
    {% for review in event.reviews.all %}
        <div>
            <p><strong>{{ review.user.username }}</strong> – Оценка: {{
review.rating }}/5</p>
            <p>{{ review.comment }}</p>
            <p><em>{{ review.created_at|date:"d.m.Y H:i" }}</em></p>
        </div>
    {% endfor %}
{% else %}
    <p>Нет отзывов о данном мероприятии.</p>
{% endif %}

<!-- Ссылки для редактирования и удаления (если пользователь
авторизован) -->
    <a href="{% url 'events:edit_event' event.id %}">Редактировать</a> |
    <a href="{% url 'events:delete_event' event.id %}">Удалить</a>
{% endblock %}

```

Пояснение:

- **Отображаем детали мероприятия:** Используем переменные `event` и его поля.
- **Отображаем отзывы:** Проверяем, есть ли отзывы, и выводим их.
- **Добавляем ссылки на редактирование и удаление:** Отображаются только для авторизованных пользователей.

3.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `event_detail`:

```

# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
]

```

Задача 4: Реализация операции обновления (Update)

4.1. Создание представления `edit_event`

Шаги:

1. Откройте файл `views.py`.
2. Импортируйте форму `EventForm`, если еще не сделали этого:

```
# events/views.py

from .forms import EventForm
```

3. Добавьте функцию `edit_event`:

```
# events/views.py

@login_required
def edit_event(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    if request.method == 'POST':
        form = EventForm(request.POST, instance=event)
        if form.is_valid():
            form.save()
            messages.success(request, 'Мероприятие успешно обновлено.')
            return redirect('events:event_detail', event_id=event.id)
    else:
        form = EventForm(instance=event)
    return render(request, 'events/edit_event.html', {'form': form, 'event': event})
```

Пояснение:

- `instance=event`: Позволяет загрузить существующие данные мероприятия в форму.
- При сохранении: Обновляет существующий объект, а не создает новый.

4.2. Создание шаблона `edit_event.html`

Шаги:

1. Создайте файл `edit_event.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/edit_event.html -->

{% extends 'events/base.html' %}

{% block title %}Редактировать мероприятие{% endblock %}

{% block content %}
    <h1>Редактировать мероприятие: {{ event.title }}</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Сохранить изменения</button>
    </form>
    <a href="{% url 'events:event_detail' event.id %}">Назад к
мероприятию</a>
{% endblock %}
```

4.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `edit_event`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/<int:event_id>/edit/', views.edit_event,
name='edit_event'),
]
```

Задача 5: Реализация операции удаления (Delete)

5.1. Создание представления `delete_event`

Шаги:

1. Откройте файл `views.py`.
2. Импортируйте необходимые модули:

```
# events/views.py
```



```
from django.views.generic import DeleteView
from django.urls import reverse_lazy
```

3. Добавьте класс `EventDeleteView`:

```
# events/views.py

class EventDeleteView(DeleteView):
    model = Event
    template_name = 'events/delete_event.html'
    success_url = reverse_lazy('events:event_list')

    def dispatch(self, request, *args, **kwargs):
        if not request.user.is_authenticated:
            # return redirect('login')
        return super().dispatch(request, *args, **kwargs)
```

Пояснение:

- **Используем класс `DeleteView`**: Предоставляет готовую реализацию для удаления объектов.
- `success_url`: Указывает, куда перенаправить пользователя после успешного удаления.
- **Переопределяем `dispatch`**: Для проверки авторизации пользователя.

5.2. Создание шаблона `delete_event.html`

Шаги:

1. Отредактируйте файл `delete_event.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/delete_event.html -->

{% extends 'events/base.html' %}

{% block title %}Удалить мероприятие{% endblock %}

{% block content %}
    <h1>Удалить мероприятие</h1>
    <p>Вы уверены, что хотите удалить мероприятие "{{ object.title }}"?</p>
    <form method="post">
        {% csrf_token %}
        <button type="submit">Да, удалить</button>
        <a href="{% url 'events:event_detail' object.id %}">Отмена</a>
    </form>
{% endblock %}
```

```
</form>
{% endblock %}
```

5.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `EventDeleteView`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/<int:pk>/delete/', views.EventDeleteView.as_view(),
         name='delete_event'),
]
```

Пояснение:

- Используем `pk` вместо `event_id`: Классовые представления ожидают параметр `pk` по умолчанию.

Задача 6: Добавление категорий

6.1. Создание дополнительного приложения

Поскольку категории нами еще не созданы, добавим их автоматически с помощью дополнительного небольшого приложения.

Измените файл `events/apps.py` следующим образом:

```
# events/apps.py

import os
from django.apps import AppConfig

class EventsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'events'

    def ready(self):
        print("Метод ready() вызван")
        if os.environ.get('RUN_MAIN', None) != 'true':
            # Избегаем выполнения кода при каждом автоперезагрузке сервера
```

```

        print("RUN_MAIN не равен 'true', метод ready() не будет
выполняться полностью")
        return
    else:
        print("RUN_MAIN равен 'true', продолжаем выполнение метода
ready()")

from django.db.utils import OperationalError, ProgrammingError
from django.core.exceptions import ObjectDoesNotExist
from django.db import connections
from .models import Category

try:
    # Проверяем, применены ли миграции
    db_conn = connections['default']
    db_conn.ensure_connection()
    if not Category.objects.exists():
        print("Категории не существуют, создаём их")
        Category.objects.create(name='Музыка',
description='Мероприятия, связанные с музыкой и концертами.')
        Category.objects.create(name='Искусство',
description='Выставки, галереи и другие художественные события.')
        Category.objects.create(name='Спорт',
description='Спортивные мероприятия и соревнования.')
    else:
        print("Категории уже существуют")
except (OperationalError, ProgrammingError) as e:
    # База данных еще не создана или миграции не применены
    print(f"Ошибка при попытке создать категории: {e}")
    pass

```

В файле `events/__init__.py` **добавьте:**

```

# events/__init__.py

default_app_config = 'events.apps.EventsConfig'

```

Важно: В Django 3.2 и выше это не требуется, но в некоторых случаях может помочь.

6.2. Примените все миграции

```

python manage.py makemigrations
python manage.py migrate

```

Перезапустите сервер разработки

Остановите сервер, если он запущен, и запустите его снова:

```
python manage.py runserver
```

6.4. Проверьте вывод в консоли

- Обратите внимание на сообщения, которые выводятся в консоли.

Пример ожидаемого вывода:

```
Метод ready() вызван RUN_MAIN равен 'true', продолжаем выполнение метода  
ready() Категории не существуют, создаём их`
```

Если вы не видите этих сообщений, значит метод `ready` не выполняется или не достигает этих строк кода.

Задача 7: Запуск и проверка приложения

7.1. Запуск сервера разработки

```
python manage.py runserver
```

7.2. Проверка работы приложения

- **Перейдите на страницу добавления мероприятия:**
 - <http://127.0.0.1:8000/events/add/>
- **Создайте новое мероприятие**, заполнив форму.
- **Проверьте страницу деталей мероприятия:**
 - Убедитесь, что отображаются все данные мероприятия и отзывы (если есть).
- **Отредактируйте мероприятие:**
 - Перейдите по ссылке "Редактировать" и измените данные.
- **Удалите мероприятие:**
 - Нажмите на ссылку "Удалить" и подтвердите удаление.

6.3. Устранение возможных проблем

- **Если возникают ошибки**, проверьте следующее:
 - Правильность импортов в файлах `views.py` и `urls.py`.
 - Наличие всех необходимых шаблонов в правильных директориях.
 - Корректность названий маршрутов и их использование в шаблонах.
-

Задача 7: Практическое упражнение

Задание для студентов:

- **Создайте свое собственное мероприятие**, используя форму добавления.
- **Просмотрите** его детали на соответствующей странице.
- **Отредактируйте** мероприятие, изменив некоторые данные.
- **Удалите** мероприятие, убедившись, что оно исчезло из списка.

Цель упражнения:

- Закрепить навыки работы с CRUD-операциями.
- Понять, как пользователь взаимодействует с приложением для управления данными.

Дополнительное задание

1. Реализация операции массового удаления

Цель: Добавить функциональность для массового удаления нескольких мероприятий через админ-панель.

Шаги:

1. Откройте файл `events/admin.py`.
2. Добавьте функцию для массового удаления:

```
# events/admin.py

from django.contrib import admin
from .models import Event

@admin.action(description='Удалить выбранные мероприятия')
def delete_selected_events(modeladmin, request, queryset):
    queryset.delete()

@admin.register(Event)
class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'category')
    actions = [delete_selected_events]
```

Пояснение:

- Создаем пользовательское действие `delete_selected_events`, которое удаляет выбранные мероприятия.
- Добавляем действие в список доступных действий в админ-панели.

2. Добавление подтверждения при удалении

Цель: Реализовать всплывающее окно подтверждения перед удалением мероприятия с помощью JavaScript.

Шаги:

1. Обновите ссылку на удаление в шаблоне `event_detail.html`:

```
<!-- events/templates/events/event_detail.html -->

<!-- ... ваш существующий код ... -->
<a href="{% url 'events:edit_event' event.id %}">Редактировать</a> |
<a href="{% url 'events:delete_event' event.id %}" onclick="return
confirm('Вы уверены, что хотите удалить это мероприятие?');">Удалить</a>
```

Пояснение:

- Добавляем обработчик `onclick`: При нажатии на ссылку появится диалоговое окно с подтверждением.

3. Создание представления для импорта/экспорта данных

Цель: Реализовать возможность импорта и экспорта списка мероприятий в формате CSV.

Шаги:

1. Создание представления для экспорта:

```
# events/views.py

import csv
from django.http import HttpResponse

def export_events_csv(request):
    events = Event.objects.all()
    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = 'attachment; filename="events.csv"'

    writer = csv.writer(response)
    writer.writerow(['Title', 'Description', 'Date', 'Location',
'Category'])
```

```
    for event in events:
        writer.writerow([event.title, event.description, event.date,
event.location, event.category.name])

    return response
```

2. Добавление маршрута в `urls.py` :

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/export/csv/', views.export_events_csv,
name='export_events_csv'),
]
```

3. Добавление ссылки на экспорт в шаблон `event_list.html` :

```
<!-- events/templates/events/event_list.html -->

<a href="{% url 'events:export_events_csv' %}">Экспортировать в CSV</a>
```

Пояснение:

- **Представление** `export_events_csv` : Генерирует CSV-файл со списком мероприятий.
- **Ссылка на экспорт** : Позволяет пользователю скачать файл.

Заключение

Поздравляем! Вы успешно:

- Реализовали основные CRUD-операции с базой данных в Django.
- Научились создавать формы и обрабатывать запросы в представлениях.
- Поняли, как работать с шаблонами для отображения данных пользователю.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Как обеспечить безопасность при выполнении CRUD-операций?**
 - Проверять права доступа пользователя.
 - Использовать декораторы `@login_required` и проверять авторство.
 - Обрабатывать возможные исключения и ошибки.
- **Почему важно использовать CSRF-токены в формах?**
 - Для защиты от CSRF-атак, когда злоумышленник пытается выполнить действие от имени пользователя без его ведома.
- **Как можно улучшить пользовательский интерфейс при работе с CRUD-операциями?**
 - Использовать JavaScript для динамического обновления страницы.
 - Применять фреймворки CSS, такие как Bootstrap, для улучшения дизайна.
 - Реализовать AJAX-запросы для обновления данных без перезагрузки страницы.

Полезные ресурсы

- [Документация Django: Работа с формами](#)
- [Документация Django: Представления на основе классов](#)
- [Django CRUD Tutorial](#)
- [Django Messages Framework](#)